SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I M.Kushal(1924260243)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M.kushal*

.

1. AI-Powered Drones Delivering Emergency Medicines

i. Greedy Best-First Search: behaviour, strengths and weaknesses

Greedy Best-First Search selects the next location with the smallest heuristic value, h(n). In this scenario, h(n) can be the straight-line distance from the drone's current position to the medicine delivery point. The drone therefore tends to move toward the destination that appears closest.

Its main strength is speed: it often finds a route quickly and is useful when a decision is urgent. However, it considers the heuristic rather than the full route cost. It may choose a short-looking path that contains a blocked road, severe weather, unsafe terrain, or a costly detour. With partial updates, its chosen route may become unsuitable.

ii. How A* balances path cost and heuristic

A* evaluates each candidate node using $f(n) = g(n) + h(n)$. Here, g(n) is the known cost from the starting point to node n, including distance, weather, terrain, delay, and risk when these are represented in the cost model. The heuristic h(n) estimates the remaining cost to the destination. A* balances the cost already incurred with the estimated cost remaining, rather than choosing only the apparently nearest node.

If the heuristic never overestimates the true remaining cost and the search uses suitable cost assumptions, A* can find an optimal path in a static problem. In this dynamic scenario, it must be updated or run again when new information changes edge costs or reveals blocked paths.

iii. Impact of heuristic accuracy

A useful heuristic guides the search toward promising routes and can reduce the number of nodes explored. If the straight-line distance is a reasonable estimate, it can guide both algorithms effectively. If it ignores storms, restricted airspace, terrain-related energy use, or risk, it may mislead the search. Greedy Best-First Search is especially sensitive because it relies on h(n) alone. A* also uses h(n), but g(n) helps it account for the actual cost already observed.

iv. Effect of blocked paths and dynamic changes

When a path becomes blocked, the route graph changes. The drone must mark that path unavailable, update affected costs, and reconsider its route. A previously selected route may need to be abandoned. Replanning takes computation and communication time, so frequent updates can reduce performance. The drone should use reliable live sensing and safe fallback behaviour when information is incomplete.

v. Recommendation

A* is the better baseline when route quality, delay, and safety all matter, because it accounts for both accumulated cost and estimated remaining cost. The cost function should include penalties for unsafe weather, energy consumption, and hazardous routes, not distance alone. The system should replan when significant updates arrive and reject routes that violate safety constraints. If the environment changes very frequently, an incremental replanning method such as D* Lite may be considered, subject to the system's implementation and verification. Greedy search may be faster in some cases, but speed alone is not enough for emergency medicine delivery.

2. Smart-City Traffic Signal Optimization

Formulation as an optimization problem and why traditional search is inefficient

The decision variables can represent the green-light duration and phase order at each intersection over a planning period. The objective is to minimize a weighted cost such as $J = \alpha W + \beta F + \gamma C$, where W is total vehicle waiting time, F is fuel consumption, C is congestion, and α, β, and γ express the city's priorities. Constraints include minimum and maximum green times, safe phase transitions, pedestrian crossing times, coordination between intersections, and emergency-vehicle priority.

The search space grows rapidly because each intersection has several possible signal settings and the choices interact across the road network. A basic search may evaluate too many combinations, while a fixed plan cannot respond well to changing traffic. A practical system uses traffic measurements, a limited planning horizon, and repeated optimization.

i. Hill Climbing and its limitations

Hill Climbing begins with a current signal plan, generates nearby plans (for example, changing a green duration slightly), evaluates them, and moves to a neighbouring plan with a better objective value. It repeats until no tested neighbour improves the result. It is simple and can improve a plan quickly, but it may stop at a locally good solution, depends on its starting plan, and can struggle when many intersections must be coordinated.

ii. Local maxima, plateaus and ridges

A local optimum occurs when nearby changes do not improve the plan even though a better plan exists elsewhere. For example, improving one intersection may increase queues at the next intersection, so a coordinated change is needed. A plateau is a region where several nearby plans have almost identical scores, making it difficult to identify a direction of improvement. A ridge is a narrow path of improvement that may require coordinated changes to multiple signal timings; changing only one variable at a time may fail to follow it.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse neighbouring solution, especially early in the search, with a probability controlled by a temperature parameter. The temperature is gradually reduced, so the algorithm becomes less willing to accept worse moves over time. This helps it escape local optima, although results depend on the cooling schedule and it may require careful tuning.

A Genetic Algorithm maintains a population of candidate signal plans. It evaluates each plan, selects stronger candidates, and creates new plans through crossover and mutation. Diversity can explore different combinations of timings and reduce dependence on one starting plan. However, it can be computationally expensive and needs appropriate encoding, constraints, and parameter settings.

iv. Recommendation

A hybrid approach is suitable: use a Genetic Algorithm or Simulated Annealing to explore difficult coordination choices, then refine the best candidate with local search. For real-time use, optimize over a rolling time window, limit computation, and update the plan using fresh traffic data. Hard safety and pedestrian constraints must be enforced, and a safe fallback plan should remain available. A Genetic Algorithm is useful for a large, interacting search space; Simulated Annealing may be simpler when a single candidate can be adjusted efficiently.

3. Autonomous Mars Rover in Unknown Terrain

i. Why an online search agent is required

Offline search assumes the agent has a sufficiently complete model of the environment before planning. The rover does not have a complete map and can discover hazards only as it explores. An online search agent alternates between sensing, planning, acting, and learning from the results. This lets it respond to newly discovered craters, rocks, and unexpected terrain.

ii. Partial observability and dynamic-environment challenges

The rover's sensors provide only limited information, and some hazards may be hidden or difficult to classify. Sensor noise can make distance and terrain estimates uncertain. Communication delays and limited energy also restrict how much information the rover can obtain or process. The map may change as the rover observes new areas or as its estimate of terrain conditions improves. Acting on an incomplete map creates a risk of choosing an unsafe route.

iii. Building and updating an internal model

The rover combines sensor readings, camera images, position estimates, and observations from earlier movements to maintain an internal map. It can label cells or regions as traversable, blocked, or uncertain and assign estimated travel costs or risk levels. After each action, it compares expected and observed results, updates its location estimate and map, and revises its route. Uncertain areas should receive conservative cost estimates until more evidence is available.

iv. Online search compared with uninformed and informed search

Uninformed methods such as breadth-first search or uniform-cost search do not use a goal-directed heuristic; they can be useful when the model is known, but may explore many states and are less directly suited to unknown terrain without repeated model updates. Informed methods such as A* use a heuristic to guide the search and can be efficient when the map and costs are available. Online search emphasizes learning and acting while the model is incomplete. These ideas can be combined: the rover can use an online agent that applies heuristic-guided planning to its current map and replans after each new observation.

v. Recommended approach

Use an online, receding-horizon navigation strategy with heuristic-guided planning, continuous sensing, and frequent map updates. Include explicit safety margins around hazards and treat uncertain terrain cautiously. The rover should prefer a slightly longer verified route over a shorter route with poorly understood risk. It should also monitor battery, communication windows, and mission priorities. This approach is adaptable because it learns as it moves, while safety checks prevent the search algorithm from treating every low-cost route as acceptable.

4. University Exam Timetabling as a Constraint Satisfaction Problem

Formulation: variables, domains and constraints

A Constraint Satisfaction Problem (CSP) consists of variables, a domain of possible values for each variable, and constraints that specify which combinations are allowed.

Variables: for each exam, define a variable containing its time slot and assigned examination hall. If hall allocation is handled separately, use separate time and hall variables for each exam.

Domains: each exam's domain contains available time slots and eligible halls. Domains exclude slots when the required faculty or special accommodation is unavailable, and exclude halls that lack suitable capacity or accessibility.

Constraints: (1) no student can have two exams in the same time slot; (2) a hall cannot host two exams at the same time; (3) each hall must have enough capacity and meet accommodation needs; (4) required subjects must be scheduled before dependent subjects; (5) invigilators and faculty must be available; and (6) all exams must be assigned a valid slot and hall. These are hard constraints. Preferences such as avoiding consecutive exams can be treated as soft constraints and optimized after feasibility is achieved.

ii. Backtracking search

Backtracking assigns a value to one exam variable at a time. It checks whether the assignment is consistent with the constraints already assigned. If it is consistent, the algorithm continues to the next variable. If no value is possible for a variable, it returns to an earlier variable and tries another value. The process continues until a complete valid timetable is found or all possibilities are exhausted.

iii. Constraint propagation and forward checking

After assigning an exam, forward checking removes values from the domains of unassigned exams that would conflict with that choice. For example, if a student has an exam at 10 a.m., that student's other exams cannot use the same slot. If any unassigned exam has an empty domain, the algorithm detects the conflict early and backtracks rather than continuing a doomed branch. Constraint propagation reduces unnecessary search.

iv. Real-world challenges and scalable strategy

Real timetables contain many students, courses, halls, faculty members, accessibility needs, and last-minute changes. Some constraints conflict, and preferences may compete. A scalable strategy is to combine backtracking with the minimum-remaining-values heuristic (choose the exam with the fewest legal options), degree heuristic (break ties by choosing the exam that constrains the most others), and least-constraining-value ordering (try the value that removes the fewest options for other exams). Use forward checking or stronger propagation, represent hard constraints separately from soft preferences, and validate the final timetable. For very large instances, local-search or optimization methods can improve soft-constraint quality after a feasible schedule is found.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. How Minimax models the problem

Minimax represents the game as a tree of possible states. At a MAX node, the AI selects the move with the highest evaluation; at a MIN node, the opponent is assumed to select the move that produces the lowest evaluation for the AI. Terminal states are scored as wins, losses, or draws, and values are propagated upward through the tree. In a real-time strategy game, a full tree is usually too large, so Minimax is applied to a limited search depth or to a simplified decision model.

ii. Computational challenges of large game trees

If each state has an average branching factor b and the search depth is d, the number of explored nodes can grow on the order of O(b^d). Many possible actions, simultaneous events, and long-term consequences make the tree enormous. A strict response deadline means the AI cannot search every branch. The system therefore needs pruning, move ordering, an evaluation function, and a controlled search depth.

iii. Alpha-Beta pruning with reasoning

Alpha-Beta pruning improves Minimax by avoiding branches that cannot change the final decision. Alpha (α) is the best value MAX can guarantee so far; beta (β) is the best value MIN can guarantee so far. At a MAX node, α is updated when a better value is found. At a MIN node, β is updated when a lower value is found. When $\alpha \geq \beta$, the remaining branches at that node can be pruned because the opponent or the AI already has a choice at an ancestor that makes those branches irrelevant.

For example, if MAX already has an option worth 5, and another branch reaches a MIN node that finds a response worth 3, MAX will not choose that branch if MIN can force the value down to 3. Further exploration of that branch may be unnecessary. With good move ordering, Alpha-Beta can reduce the effective search substantially while returning the same Minimax decision for the searched tree.

iv. Evaluation function

A practical heuristic evaluation function for a non-terminal state could be:

$$\textbf{Eval(s)} = w_1\textbf{(Resource Advantage)} + w_2\textbf{(Unit Strength)} + w_3\textbf{(Strategic Control)} + w_4\textbf{(Objective Progress)} - w_5\textbf{(Threat/Risk)}$$

Resource advantage measures useful resources compared with the opponent. Unit strength estimates surviving and effective units. Strategic control evaluates important locations and map access. Objective progress measures progress toward victory conditions. Threat/risk estimates exposure to attacks and vulnerable assets. The weights w_1 to w_5 should be tuned and tested using representative games. The evaluation should be fast, consistent, and aligned with the game's actual winning conditions.

v. Balancing optimality and real-time performance

Use depth-limited Minimax with Alpha-Beta pruning, strong move ordering, and a lightweight evaluation function. Iterative deepening searches depth 1, then 2, and so on; if the time limit is reached, the AI can return the best move from the last completed depth. A transposition table can reuse results for repeated states, while selective extensions can focus extra search on critical moves. The system must always retain a legal fallback move. These methods improve decision quality under time limits, although depth-limited search is not guaranteed to find the globally optimal move in the full game.

Conclusion

The five scenarios demonstrate that AI search must be selected according to the environment and objective. A* balances known path cost with a heuristic for route planning; metaheuristic optimization can help with large traffic-control spaces; online search supports exploration with incomplete maps; CSP methods organize exam scheduling under constraints; and Minimax with Alpha-Beta pruning supports strategic game decisions under computation limits. In safety-critical applications, algorithms should be combined with current data, explicit safety constraints, validation, and reliable fallback behaviour.